Zephyr Project

IT-Schulungen.com
Technology & Transformation

Threads erzeugen und konfigurieren

Statische und dynamische Threads

2.03

Was wir in 2.03 klären

Vom Entry Point zum diagnostizierbaren Worker

01

Weg wählen

statisch oder dynamisch erzeugen

02

Entry

Funktion und drei Parameter

03

Ressourcen

Stack, Kontrollblock und Lebensdauer

04

Start

Priorität, Optionen und Verzögerung

05

Diagnose

Thread-ID und Name nutzen

Zielbild: Ein benannter Worker läuft mit bewusst gewähltem Stack neben main().

Zwei Erzeugungswege, ein Thread-Modell

01	`K_THREAD_DEFINE`	kompakt · Ressourcen statisch · automatischer Start
02	`k_thread_create()`	flexibel · explizite Objekte · Start zur Laufzeit
03	**Gemeinsames Modell**	Entry · eigener Stack · Priorität · Zustand · ID

Entscheidungskriterium: Ist Anzahl und Lebensdauer der Threads bereits beim Build bekannt?

Statisch mit K_THREAD_DEFINE

QUELLCODE

```
void worker(void *a, void *b, void *c)
{
    while (1) {
        k_sleep(K_MSEC(500));
    }
}

K_THREAD_DEFINE(worker_id, 1024, worker,
                NULL, NULL, NULL,
                5, 0, K_NO_WAIT);
```

01 Ein Makro, alles statisch

Threadobjekt und Stack werden beim Build angelegt.
Der Start ist direkt in der Definition dokumentiert.

02 Parameter im Blick

Name, Stack, Entry, Argumente und Priorität gehören zusammen.
Optionen und Startverzögerung vervollständigen die Definition.

K_THREAD_DEFINE eignet sich für dauerhaft bekannte Threads mit planbaren Ressourcen.

Dynamisch mit k_thread_create()

THREAD-ERZEUGUNG

```
K_THREAD_STACK_DEFINE(worker_stack, 1024);
static struct k_thread worker_data;

k_tid_t tid = k_thread_create(
    &worker_data,
    worker_stack,
    K_THREAD_STACK_SIZEOF(worker_stack),
    worker, NULL, NULL, NULL,
    5, 0, K_NO_WAIT);

k_thread_name_set(tid, "sensor_worker");
```

01 Stack

Speicher bleibt während der gesamten Thread-Lebensdauer gültig.

02 Kontrollblock

struct k_thread speichert Zustand und Verwaltungsdaten.

03 Thread-ID

k_thread_create() liefert k_tid_t für spätere Operationen.

Parameter einer Thread Definition

SIGNATUR

Entry + Ressourcen + Scheduling + Start

`Entry und p1–p3`	Ablauf des Threads und drei generische Argumente
`Stack`	Größe aus Call-Tiefe, lokalen Daten und Reserve ableiten
`Priorität und Optionen`	Scheduling-Klasse und Thread-Eigenschaften festlegen
`Startverzögerung`	K_NO_WAIT, Timeout oder K_FOREVER bewusst wählen

Parameter werden gemeinsam dokumentiert: Ein isolierter Zahlenwert erklärt keinen Thread.

Stack und Lebensdauer planen

CHECKLISTE

Ressourcen vor dem Start klären

Stackbedarf	Call-Tiefe, lokale Daten, Bibliotheken und Reserve
Lebensdauer	Stack und Threadobjekt dürfen nicht vor dem Thread enden
Messung	Stack-Nutzung im Test beobachten statt nur schätzen
Reserve	Fehlerpfade und spätere Erweiterungen berücksichtigen
Dokumentation	Annahme, Messwert und gewählte Größe festhalten

Zu kleine Stacks gefährden die Stabilität; unnötig große Stacks verschwenden knappen RAM.

Startzeit, ID und Name

STEUERUNG

```
k_tid_t tid = k_thread_create(
    &worker_data, worker_stack,
    K_THREAD_STACK_SIZEOF(worker_stack),
    worker, NULL, NULL, NULL,
    5, 0, K_FOREVER);

k_thread_name_set(tid, "sensor_worker");
k_thread_start(tid);
```

01 Startzeit

K_NO_WAIT startet sofort; ein Timeout verzögert; K_FOREVER wartet auf k_thread_start().

02 Diagnose

k_tid_t speichern und einen sprechenden Namen für Logs und Thread-Analyse vergeben.

Startverhalten und Diagnoseinformationen gehören zur Threaddefinition, nicht zur Nacharbeit.

Kurzcheck: Thread-Erzeugung

BAUSTEINE

Sicher zuordnen

Entry-Funktion

Threadobjekt und Stack

Priorität und Optionen

Startzeit und Thread-ID

ENTSCHEIDUNG

Statisch oder dynamisch?

bekannte Anzahl: K_THREAD_DEFINE

Laufzeitbedarf: k_thread_create()

Ressourcen bleiben gültig

Stackgröße ist begründet

ZIELBILD

Worker läuft neben main()

eindeutige Ausgabe

sprechender Name

gespeicherte Thread-ID

Lernziel: Erzeugungsweg, Stack-Lebensdauer, Startzeit und Diagnosezugriff verstehen.